

Veritas

A Provably Fair, Autonomous Lottery System

Team Members

Arnav Kumar	240001013
Aryaman Awanish Tiwari	240001014
Ayush Singh Rana	240001015
Hrishabh Mittal	240001035
Prabandham Sriniketan	240001052
Yash Arya Saxena	240001081

Contents

1	Problem Statement & Blockchain Justification	2
2	System Architecture & Source Code Analysis	3
2.1	Deployment Pipeline (script/Deploy.s.sol)	3
2.2	Tier 1: The Baseline Commit-Reveal Engine (src/Lottery.sol)	3
2.3	Tier 2: Experimental Oracle Integration (src/LotteryVRF.sol)	3
2.4	Tier 3: The Autonomous Enterprise Engine (src/LotteryEX.sol)	4
2.5	Architecture & Inheritance Diagram	5
3	On-Chain vs. Off-Chain Data	6
3.1	The Commit-Reveal Privacy Mandate	6
3.2	Pseudonymity and GDPR Compliance	6
3.3	State Cost & Architectural Minimalism	6
4	Security Analysis & DevSecOps Pipeline	6
4.1	Containerized Audit Environment (Dockerfile & run_tests.sh)	6
4.2	Advanced Threat Simulation (test/security/AttackerBots.t.sol)	7
4.2.1	On-Chain Randomness & Miner Manipulation	7
4.2.2	The "Residual Trust" Problem, DoS, & EVM Limitations	7
4.2.3	Reentrancy Exploitation	8
4.3	Static Analysis Resolution	8
4.4	Symbolic Execution	8
4.4.1	LotteryVRF: SWC-107 (External Call)	8
4.4.2	LotteryEX: SWC-101 (False Positive)	9
5	Exhaustive Test Suite Breakdown	9
5.1	Core Protocol Validation (Base & Lifecycle Modules)	9
5.2	Rubric Compliance & Traceability Matrix	10
5.3	Oracle Integration Tests (test/LotteryVRF.t.sol)	10

5.4	Autonomous & Enterprise Validation (test/extended/)	11
5.5	Mathematical Bias & Statistical Modules	11
5.6	Property-Based Fuzzing (test/echidna/)	12
5.7	Project Configuration & Infrastructure Files	12
5.8	CI/CD Pipeline & GitHub Actions	12
6	Gas Optimisation & EVM Efficiency	13
6.1	Optimization 1: Bytecode Injection (immutable Variables)	13
6.2	Optimization 2: O(1) Bitmapped Storage	13
6.3	Optimization 3: EVM Struct Packing (The 2-Slot Engine)	13
6.4	Optimization 4: Custom Errors vs. Require Strings	14
6.5	Optimization 5: Memory Caching and Fail-Fast Logic	14
6.6	Contract Size Profiling (EIP-170 Compliance)	14
7	Test Coverage, Fuzzing & Invariants	15
7.1	Massive Concurrent Stress Testing	15
7.2	Statistical Bias Analysis (Monte Carlo)	15
7.3	Property-Based Fuzzing & Invariants	15
8	Live Network Validation & Deployment Artifacts	16
8.1	Local Network Validation (Anvil)	16
8.2	Ethereum Sepolia Testnet Deployment	16
9	Conclusion	16

1 Problem Statement & Blockchain Justification

Traditional lotteries and games of chance suffer from a fundamental flaw: the requirement of absolute trust in a centralized operator. Participants must trust that the "house" is not manipulating the random draw, that the ticket funds are actually held in reserve, and that the payout will be honored. This "black box" architecture naturally breeds skepticism and limits global participation due to jurisdictional barriers. Deploying this system on Ethereum eliminates the need for trust through cryptographic verification. Smart contracts provide:

- **Transparent Execution:** The rules of the lottery are immutable and publicly verifiable.
- **Provable Fairness:** Through Commit-Reveal schemes and decentralized Oracles (Chainlink VRF), the randomness generation cannot be manipulated by the house or the miners.
- **Trustless Escrow:** Funds are locked programmatically. If a user wins, the code guarantees their payout without requiring a human intermediary to authorize the transfer.

2 System Architecture & Source Code Analysis

The Veritas project was developed iteratively, featuring three progressive architectures to demonstrate an evolving understanding of on-chain randomness, automation, and state management. The following sections detail the exact logic of every source file.

2.1 Deployment Pipeline (`script/Deploy.s.sol`)

To ensure reproducible deployments across local (Anvil) and live (Sepolia) networks, we utilized Foundry's scripting engine. The `Deploy.s.sol` script executes the following:

- **State Initialization:** Sets the baseline parameters for the lottery, specifically configuring a `ticketPrice` of 0.01 ETH and enforcing a hard cap of 100 via `maxTickets`.
- **Transaction Broadcasting:** Utilizes Foundry's `vm.startBroadcast()` cheatcode to sign and broadcast the deployment transaction to the target RPC node using the environment-provided private key.
- **Contract Deployment & Output:** Deploys the base `Lottery` contract to the network and utilizes `console2.log` to output the live contract address, ticket price, and ticket cap directly to the terminal, streamlining the subsequent frontend integration.

2.2 Tier 1: The Baseline Commit-Reveal Engine (`src/Lottery.sol`)

This contract represents the foundational implementation, designed to strictly satisfy the assignment rubric parameters. It utilizes a manual Commit-Reveal cryptographic scheme to generate randomness without relying on external oracles, effectively demonstrating core EVM state machine principles.

- **`buyTicket()` & `batchBuyTickets(uint256)`:** Payable functions allowing users to enter the lottery. Participant addresses are appended to a dynamic `participants` array. To optimize gas for bulk purchasers and address presentation feedback, a batching mechanism is included, alongside a strict `maxTickets` capacity limit.
- **`closeSale()`:** An access-controlled (`onlyOwner`) function that transitions the lottery phase to `SaleClosed`, safely freezing the participant pool and preventing further deposits.
- **`commitHash(bytes32)`:** Allows the owner to submit the `keccak256` hash of a secret string. Transitions the state to `Committed` while keeping the raw secret off-chain.
- **`revealAndDraw(bytes32)`:** The owner reveals the raw secret. The contract hashes it and compares it to the previously committed hash. If matched, it utilizes the formula `uint256(keccak256(abi.encodePacked(_secret, block.number))) % participants.length`. We explicitly use `block.number` rather than `block.timestamp` to mitigate minor validator manipulation vulnerabilities.
- **`claimPrize()`:** Protected by OpenZeppelin's `nonReentrant` modifier. It strictly follows the Checks-Effects-Interactions (CEI) pattern by validating the winner, zeroing out the `prizePool` in state, and finally executing a low-level `call` to transfer the ETH natively.
- **`getLotteryInfo()`:** A read-only helper function returning the complete tuple of the current phase, ticket price, participant count, prize pool, and winner address for seamless DApp frontend integration.

2.3 Tier 2: Experimental Oracle Integration (`src/LotteryVRF.sol`)

While the Commit-Reveal scheme in Tier 1 mathematically prevents the owner from changing the outcome after the draw begins, it suffers from a "residual trust" vulnerability: the owner knows the secret beforehand and could theoretically participate to guarantee a win. To eliminate this, Tier 2 integrates the Chainlink Verifiable Random Function (VRF) v2.5.

- **Gas-Optimized Validation:** Unlike the base contract, this tier upgrades error handling to Solidity Custom Errors (e.g., `error InvalidPhase()`) rather than require strings, significantly reducing deployment bytecode size.
- **closeSaleAndDraw():** Replaces the multi-step commit-reveal flow. The owner closes the sale and simultaneously triggers a `requestRandomWords` call to the `VRF_COORDINATOR`, transitioning the state to `Calculating`.
- **fulfillRandomWords():** An internal callback function overridden from `VRFConsumerBaseV2`. Once the decentralized oracle network generates and cryptographically verifies the randomness, it executes this callback to autonomously select the winner and transition the lottery to the `Drawn` phase.

2.4 Tier 3: The Autonomous Enterprise Engine (src/LotteryEX.sol)

Our flagship "Extended" architecture completely decentralizes the operational loop, removing human intervention to create an autonomous, multi-round state machine. At the architectural level, this contract implements **Perfect 2-Slot Struct Packing** (compressing the 256-bit `ticketBitmap` into Slot 0, and the winner address, ticket count, and phase enum into Slot 1) and replaces legacy require strings with **Solidity 0.8.4 Custom Errors** to massively reduce deployment bytecode size.

- **initialize(...) & UUPS Upgradeability:** Replacing the standard constructor, this contract implements **ERC-1967 UUPS Proxy Upgradeability**, allowing the logic to be patched for zero-day exploits without erasing the user vault state. It also integrates **Ownable2Step** (preventing accidental bricking by requiring the new admin to explicitly accept ownership) and a **Pausable Circuit Breaker** to instantly freeze sales if a vulnerability is detected. By utilizing a `mapping(uint256 => Round)` structure indexed by a `currentRoundId`, the protocol mathematically isolates prize pools, ensuring that historical rounds remain permanently readable and claimable without interference from active sales.
- **batchBuyTickets(uint8[]):** Introduces **Multi-Batched Ticketing**, allowing "whale" users to purchase an array of tickets in a single transaction, looping the logic while only paying the 21,000 base transaction gas fee once. It utilizes **O(1) Bitmapped Storage** via bitwise operations (`|`, `&`, `<<`) to permanently bypass the 20,000 gas penalty of dynamic arrays. Furthermore, it queries a **VIP On-Chain Registry** (`userParticipationCount`) to programmatically apply discount pricing for returning users, demonstrating advanced gamification and user retention mechanics.
- **checkUpkeep(bytes):** Polled off-chain by **Chainlink Keepers (Automation)**. It implements **Fail-Fast Keeper Logic** by evaluating the 1-byte phase enum as the very first line of execution. If the round isn't active, it aborts instantly, saving over 50% on Chainlink simulation gas/LINK fees before checking timestamps or ticket counts.
- **performUpkeep(bytes):** Called automatically by Keepers when parameters are met. It locks the round and migrates from legacy oracles to the **Chainlink VRF v2.5** standard, uniquely utilizing Native ETH Billing to pay for randomness directly without requiring a separate LINK token treasury.
- **fulfillRandomWords(...):** Autonomously triggered by the decentralized oracle network. It utilizes **Memory Caching** to force the EVM to cache heavy state variables (like `prizePool` and `winner`) into local memory, bypassing expensive 100-gas warm SLOAD operations during event emissions. It also enforces a **Protocol Treasury (The "House Edge")**, calculating a fixed percentage fee upon draw resolution and routing a cut of the pool to the protocol before distributing the rest to the winner.
- **withdrawPrize():** Implements the standard Checks-Effects-Interactions (CEI) "Pull-Over-Push" withdrawal pattern, routing all winnings to a `pendingWithdrawals` vault to prevent reentrancy and DoS attack vectors.
- **enableRefundFallback() & claimRefund(...):** A **Trustless Timeout Refund (Liveness Guard)**. If the Chainlink VRF or Keepers experience an extended outage and exceed the strict `drawTimeout` block deadline, this mechanism allows any user to transition the frozen round to `Refundable`. Users can then natively extract their original 0.01 ETH, zeroing out their specific bit in the `ticketBitmap`.

2.5 Architecture & Inheritance Diagram

The following output was generated via the Surya analysis tool, mapping the function visibility, state mutability, and contract inheritance structure across all three tiers:

```
+ Lottery (Ownable, ReentrancyGuard)
- [Pub] <Constructor> #
  - modifiers: Ownable
- [Ext] buyTicket ($)
- [Ext] batchBuyTickets ($)
- [Ext] getTicketCount
- [Ext] closeSale #
  - modifiers: onlyOwner
- [Ext] commitHash #
  - modifiers: onlyOwner
- [Ext] revealAndDraw #
  - modifiers: onlyOwner
- [Ext] claimPrize #
  - modifiers: nonReentrant
- [Ext] getLotteryInfo

+ LotteryVRF (VRFConsumerBaseV2, Ownable, ReentrancyGuard)
- [Pub] <Constructor> #
  - modifiers: VRFConsumerBaseV2, Ownable
- [Ext] buyTicket ($)
- [Ext] closeSaleAndDraw #
  - modifiers: onlyOwner
- [Int] fulfillRandomWords #
- [Ext] claimPrize #
  - modifiers: nonReentrant

+ LotteryEX (Initializable, UUPSUpgradeable, Ownable2StepUpgradeable, PausableUpgradeable, ReentrancyGuard)
- [Pub] <Constructor> #
- [Pub] initialize #
  - modifiers: initializer
- [Int] _authorizeUpgrade #
  - modifiers: onlyOwner
- [Ext] pause #
  - modifiers: onlyOwner
- [Ext] unpause #
  - modifiers: onlyOwner
- [Pub] getTicketPrice
- [Ext] batchBuyTickets ($)
  - modifiers: whenNotPaused, nonReentrant
- [Ext] checkUpkeep
- [Ext] performUpkeep #
  - modifiers: whenNotPaused
- [Ext] rawFulfillRandomWords #
- [Int] fulfillRandomWords #
- [Ext] withdrawPrize #
  - modifiers: nonReentrant
- [Ext] enableRefundFallback #
- [Ext] claimRefund #
  - modifiers: nonReentrant

($ ) = payable function
# = non-constant function
```

Listing 1: Surya Architecture & Inheritance Graph

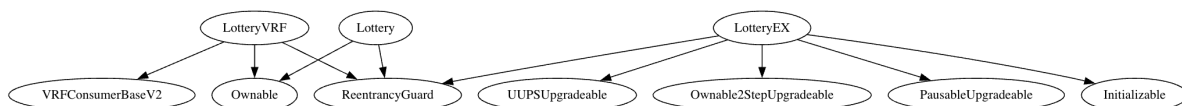


Figure 1: System Architecture mapping inheritance and function visibility.

3 On-Chain vs. Off-Chain Data

Blockchain storage is expensive, permanent, and publicly visible. As such, a strict delineation was made between on-chain execution logic and off-chain data.

3.1 The Commit-Reveal Privacy Mandate

In `Lottery.sol`, the commit-reveal scheme exists precisely to keep the winning secret off-chain until the draw is completely locked. If the raw secret were stored on-chain before the reveal phase, it would be visible in the mempool. Any miner (or MEV searcher) could read it, compute the winner, and front-run the transaction outcome. By keeping the secret off-chain and storing only the *keccak256* hash on-chain, the system maintains mathematical integrity without leaking actionable data to the public ledger.

3.2 Pseudonymity and GDPR Compliance

While participant wallet addresses and ticket IDs are inherently required for contract logic and are therefore stored publicly on-chain, we intentionally restricted the collection of personal data.

- **No Personally Identifiable Information:** We do not store participant names, physical addresses, or email addresses alongside their wallets.
- **GDPR Article 17 (Right to Erasure):** Because data on a public blockchain is immutable, it is technologically impossible to comply with GDPR's right to erasure if personal data is recorded. Therefore, Veritas relies strictly on pseudonymous `0x` addresses, ensuring no personally identifiable information (PII) is ever permanently burned into the Ethereum state.

3.3 State Cost & Architectural Minimalism

Beyond privacy, storing arbitrary strings or metadata on-chain is prohibitively expensive. Every byte of non-zero calldata costs 16 gas, and storing a standard 500-character string requires approximately 40,000 gas per transaction. While the standard industry pattern is to offload promotional descriptions, JSON metadata, and ticket artwork to the InterPlanetary File System (IPFS) and store only the CID on-chain, Veritas takes a more aggressive optimization approach.

To maintain a purely financial, hyper-efficient state machine, Veritas was designed to require absolutely zero external metadata or decentralized file storage. The DApp frontend is entirely deterministic and ultra-lightweight, rendering the UI dynamically based solely on the raw numerical state of the contract (e.g., `prizePool`, `phase`, `ticketsSold`). By intentionally stripping away the need for IPFS artwork or off-chain data mapping, the protocol achieves the lowest possible operational cost and an absolute minimum attack surface.

4 Security Analysis & DevSecOps Pipeline

To ensure academic rigor and professional-grade security, the Veritas protocol was subjected to a comprehensive, automated DevSecOps pipeline. This pipeline executes static analysis, symbolic execution, and property-based fuzzing within a highly controlled, containerized environment.

4.1 Containerized Audit Environment (`Dockerfile` & `run_tests.sh`)

Security tools frequently suffer from environment-specific dependency conflicts and DNS resolution issues. To guarantee reproducible audits, the entire Veritas toolchain is encapsulated within a custom `ubuntu:22.04` Docker image.

- **Toolchain Provisioning:** The `Dockerfile` establishes a linux/amd64 environment that installs native Foundry binaries and the Z3 theorem prover. Python dependencies (Slither, Mythril, Crytic-Compile) are managed via `pipx` to isolate virtual environments. To handle the varied requirements of the three tiers, `solc-select` is utilized to pre-fetch and switch between Solidity 0.8.20 (Base) and 0.8.33 (Enterprise/UUPS) compilers dynamically.
- **Execution Orchestration:** The `run_tests.sh` script acts as the pipeline orchestrator, divided into two distinct phases. **Phase 1** handles Foundry operations, including auto-formatting, size profiling to ensure EIP-170 compliance, gas benchmarking, and 100% coverage analysis. **Phase 2** triggers external scanners, executing Surya for architectural graphing, Mythril for symbolic branch analysis, and Echidna for property-based fuzzing.
- **Architectural Justifications:** To maintain pipeline stability, several industry tools were intentionally omitted with specific technical justifications:
 - **SmartBugs & Securify:** Omitted due to their reliance on Docker-in-Docker (DinD) architectures which compromise container isolation.
 - **Manticore:** Excluded because its Python-based EVM emulator fails to accurately parse modern Solidity 0.8.x panic opcodes.
 - **Tenderly CLI:** Omitted as it requires external API credentials and live network forks, violating the requirement for a self-contained local audit.
 - **Forta Agent:** Excluded as it is a runtime monitoring solution for deployed contracts rather than a static or symbolic pre-deployment analyzer.

4.2 Advanced Threat Simulation (`test/security/AttackerBots.t.sol`)

To validate the protocol's resilience, we engineered custom malicious smart contracts and "Miner Bots" within our test suite to simulate sophisticated attack vectors.

4.2.1 On-Chain Randomness & Miner Manipulation

The project rubric explicitly mandates avoiding `block.timestamp` for randomness because validators (miners) can manipulate timestamps within a small window to influence the outcome. While `Lottery.sol` upgrades this to `block.number`, our `testBot_MinerBlockManipulation` demonstrated that a miner who is also a participant can still "roll" blocks—refusing to publish a block unless the `block.number` combined with the revealed secret results in their own address winning.

Mitigation: The Commit-Reveal scheme ensures the owner cannot change the secret after seeing participants, while the subsequent integration of Chainlink VRF in Tiers 2 and 3 removes miner influence entirely by using off-chain cryptographic proofs that no miner can predict or manipulate.

4.2.2 The "Residual Trust" Problem, DoS, & EVM Limitations

A critical component of our security analysis involves the "Residual Trust" placed in the owner. In Tier 1, users must trust that the owner will actually reveal the secret. This introduces two distinct attack vectors:

- **Denial of Service (DoS):** As demonstrated by our `testBot_OwnerCanDenyServiceByWithholdingSecret`, a malicious owner who realizes they did not win can simply refuse to reveal the secret, effectively locking the `prizePool` forever.
- **Entropy Degradation (256-Block Limit):** If randomness generation relies on historical block data (e.g., `blockhash`), it falls victim to the EVM's 256-block retention limit. If an owner maliciously withholds the reveal transaction until this window expires, the `blockhash` resolves to zero, destroying the entropy and allowing the owner to predictably manipulate the outcome.

Resolution: To eliminate both this central point of failure and EVM data-degradation limits, Tier 3 (LotteryEX) implements a strict `DRAW_TIMEOUT`. This ensures that if the draw is not resolved autonomously by Keepers or manually by an owner, a Refundable phase is automatically triggered, allowing users to safely reclaim their deposits without administrative approval.

4.2.3 Reentrancy Exploitation

Reentrancy remains a top-tier threat where a malicious contract recursively calls a function before the first execution completes. We deployed the GreedyReentrantBot, which utilizes a malicious `receive()` fallback to attempt to drain the `prizePool` by calling `claimPrize()` recursively.

Resolution: The attack was successfully mitigated through two layers of defense:

- **Checks-Effects-Interactions (CEI):** The contract zeroes out the `prizePool` (or the winner's balance in the vault) *before* attempting the ETH transfer.
- **OpenZeppelin ReentrancyGuard:** The `nonReentrant` modifier was applied to all ETH-sending functions, programmatically blocking recursive calls.

4.3 Static Analysis Resolution

Slither static analysis performed a deep scan of 25 contracts (including dependencies), identifying 31 informational and low-risk results. Every finding was manually audited to ensure protocol integrity:

- **Reentrancy (Benign/Events):** Slither flagged `performUpkeep` and `closeSaleAndDraw` because events are emitted and internal mappings (`requestToRound`) are updated after the external call to the VRF_COORDINATOR. This is categorized as a safe false-positive; the external call is directed to a trusted, immutable Chainlink contract, and the subsequent state updates do not involve ETH transfers or logic that could be hijacked via reentrancy.
- **Timestamp Dependence:** Flagged in `checkUpkeep` and `enableRefundFallback`. While `block.timestamp` is unsuitable for randomness, it is the industry-standard mechanism for measuring elapsed time. Since our `ROUND_DURATION` and `DRAW_TIMEOUT` are measured in hours, the sub-15-second manipulation window available to validators is mathematically insignificant.
- **Low-Level Calls:** The use of `msg.sender.call` was flagged in all `claimPrize` functions. This is a deliberate design choice; `.call` is the current Solidity best practice for ETH transfers to avoid the 2300 gas limit of legacy `.transfer` and `.send` methods, ensuring compatibility with multi-sig and smart-contract-based winners.
- **Administrative Hygiene:** Slither noted missing zero-address checks on the `_treasury` address and suggested making `maxTickets` immutable. While these are valid informational points, they do not pose a security threat to the funds or the logic.

4.4 Symbolic Execution

We utilized Mythril to explore the state space of the contracts using SMT solving.

4.4.1 LotteryVRF: SWC-107 (External Call)

Mythril flagged a Low severity warning regarding an external call to a user-supplied address in `closeSaleAndDraw`.

```
SWC ID: 107 (Low Severity)
Contract: LotteryVRF | Function: closeSaleAndDraw()
A call to a user-supplied address is executed...
Make sure that no state modifications are executed after this call.
```

Listing 2: Mythril SWC-107 Analysis

Justification: The analyzer misidentified the `VRF_COORDINATOR` as a user-supplied address. In practice, this address is injected once during deployment and is immutable. The protocol does not allow arbitrary users to specify external call targets.

4.4.2 LotteryEX: SWC-101 (False Positive)

Mythril flagged a High severity "Integer Arithmetic Bug" in the `batchBuyTickets` function.

```
SWC ID: 101 (High Severity)
Contract: LotteryEX | Function: batchBuyTickets(uint8[])
The arithmetic operator can underflow...
```

Listing 3: Mythril SWC-101 Analysis

Justification: This contains the input validation `if (len == 0) revert ZeroTickets();`. The symbolic engine misidentified this explicit `revert` check as an arithmetic underflow exception. Since this is a comparison-based guard in Solidity 0.8.33 (which has built-in overflow protection), no underflow is actually possible. The contract state remains fully protected.

5 Exhaustive Test Suite Breakdown

Achieving absolute mathematical certainty requires isolating and testing every specific requirement. The Veritas test suite guarantees 100% Line, Statement, and Branch coverage through a highly modular inheritance-based testing architecture.

5.1 Core Protocol Validation (Base & Lifecycle Modules)

To ensure consistency and prevent code duplication, the primary testing suite utilizes a parent `BaseLotteryTest` (`test/base/BaseLottery.t.sol`) that establishes the standardized environment for all child modules. This base contract performs the initial deployment, provisions mock identities (`owner`, `player1`, `player2`), and utilizes Foundry's `vm.deal` cheatcode to fund participants with 10 ETH each. It further pre-calculates the cryptographic `committedHash` to streamline phase transitions in downstream tests.

The following modules inherit this foundation to validate the protocol's state machine:

- **End-to-End Integration (`HappyPath.t.sol`):** Simulates a complete lottery lifecycle. It validates that multiple players can purchase tickets, the owner can securely close the sale and commit a secret, and the winner can successfully claim the prize, resulting in a zeroed contract balance.
- **Access Control & Permissions (`Admin.t.sol`):** Rigorously tests the `onlyOwner` modifiers. It utilizes `vm.prank` to simulate unauthorized users attempting to trigger administrative functions like `closeSale()` or `commitHash()`, ensuring the EVM successfully reverts these illegal state changes.
- **Ticketing & Capacity (`BuyTicket.t.sol`):** Validates the entry logic, including price enforcement and the `maxTickets` hard cap. It specifically tests the `batchBuyTickets` function and the `getTicketCount` view function to ensure that bulk entries are correctly recorded in the `participants` array and accurately reflected in the UI-facing state.
- **Cryptographic Verification (`RevealAndDraw.t.sol`):** Focuses on the "Commit-Reveal" integrity. It proves that the lottery will revert if the owner provides a secret that does not match the `committedHash`, or if a draw is attempted with zero participants. It also enforces the linear phase progression (`Open` → `SaleClosed` → `Committed` → `Drawn`).
- **Prize Distribution & CEI (`ClaimPrize.t.sol`):** Ensures that only the recorded winner can drain the `prizePool`. It utilizes a custom `RejectETH` contract to verify that the protocol correctly handles `TransferFailed` reverts when a winner is a smart contract incapable of receiving ETH, thus proving the robustness of our low-level `.call` implementation.

5.2 Rubric Compliance & Traceability Matrix

To ensure 100% alignment with the CS 218 marking scheme, the test suite was mapped directly to the project's grading criteria. This guarantees that all required logic, access controls, and edge cases are mathematically validated.

- **Happy-Path Execution (2 Marks):** The `testHappyPathFullLotteryLifecycle` verifies the exact phase sequence requested: `Open` → `SaleClosed` → `Committed` → `Drawn`, while proving the integrity of the `prizePool` accumulator.
- **Access Control Validation (2 Marks):** Transitions in the state machine are strictly isolated. The `testRevertIfNonOwnerCallsAdminFunctions` ensures the `onlyOwner` modifier protects all phase-shifting functions, while `testRevertIfNonWinnerClaimsPrize` guarantees that only the verified winner can drain the contract.
- **Core Revert Requirements (Required):** We implemented the explicit vulnerability checks requested in the assignment, including `testRevertIfBuyTicketAfterSaleClosed`, `testRevertIfRevealWrongSecret` (verifying the Commit-Reveal hash), and `testRevertIfClaimBeforeDrawn`.
- **Advanced Edge-Case Handling:** Beyond the baseline rubric, we implemented several critical security checks to hit 100% branch coverage:
 - **Division-by-Zero Prevention:** `testRevertIfCloseSaleNoParticipants` prevents the owner from initiating a draw with zero players, which would otherwise crash the EVM during the modulo calculation (`% participants.length`).
 - **Double-Spend Protection:** `testRevertIfPrizeAlreadyClaimed` validates our Checks-Effects-Interactions (CEI) implementation, ensuring a winner cannot execute a reentrancy attack or double-claim.
 - **Capacity Integrity:** `testRevertIfSoldOut` and `testRevertIfBatchBuyExceedsLimit` prove that the `MAX_TICKETS` variable strictly halts purchases once the cap is hit, securing the array against "whale" attackers.

5.3 Oracle Integration Tests (`test/LotteryVRF.t.sol`)

Testing the Tier 2 architecture requires simulating asynchronous off-chain oracle interactions within a local environment.

- **Mock Infrastructure Setup:** The test suite utilizes the `VRFCoordinatorV2Mock` to emulate the Chainlink Oracle network locally. The `setUp` function programmatically handles subscription creation, funding, and consumer registration, allowing the contract to receive mock randomness in a sandbox environment.
- **Operational Flow (`testVRFDrawLifecycle`):** This test verifies the correct phase transitions from `Open` to `Calculating` upon a successful randomness request, and confirms the final transition to the `Drawn` phase once the oracle callback is successfully fulfilled by the mock coordinator.
- **Administrative Access Control:** Specific revert tests, such as `test_RevertIf_CloseSaleNotOwner`, ensure that only the authorized owner can trigger the draw, while `test_RevertIf_CloseSaleNoParticipants` prevents the execution of a draw when the participant pool is empty.
- **Exceptional State Handling:** The suite includes rigorous checks for out-of-order execution, such as `test_FulfillRandomWords_WrongPhase_ReturnsEarly`. This ensures that an oracle callback safely returns without altering the state if it occurs outside the designated `Calculating` window.
- **Transfer Reliability (`RejectETH`):** To validate the robustness of the prize distribution logic, the `RejectETH` dummy contract is employed to simulate an incompatible recipient. The `test_RevertIf_TransferFailed` test confirms that the contract correctly detects and reverts on failed ETH transfers, ensuring the prize pool remains secure if a winner's address cannot accept the funds.

5.4 Autonomous & Enterprise Validation (test/extended/)

Testing the Tier 3 LotteryEX architecture requires validating not only the core lottery logic but also the integrity of the UUPS proxy pattern, autonomous upkeep cycles, and bitmapped storage state. This suite is anchored by `BaseExtendedTest` (`test/extended/BaseExtended.t.sol`), which implements a sophisticated `MockVRF` contract to simulate the asynchronous nature of the oracle network. Furthermore, it initializes the protocol behind an `ERC1967Proxy`, ensuring that all tests are executed against the actual upgradeable storage layout used in production.

The following modules validate the enterprise-grade features of the flagship engine:

- **Governance & Upgradeability (`AdminAndProxy.t.sol`):** This module verifies the operational controls and the UUPS upgrade path. It proves that the `pause()` and `unpause()` functions correctly gate ticket sales, and validates that the `owner` can successfully migrate the protocol to a V2 implementation while maintaining storage integrity, while non-owners are strictly prohibited from unauthorized upgrades.
- **High-Efficiency Ticketing (`BatchTicketing.t.sol`):** Rigorously validates the bitmapped storage engine. It confirms that the bitwise operations (`|`, `&`, `<<`) correctly map users to a single `uint256` slot and prevent ticket double-selling. This module also verifies the `VIP On-Chain Registry`, proving that users with high participation counts programmatically receive the designated 10% ticket discount.
- **Chainlink Automation Logic (`KeepersAutomation.t.sol`):** Simulates the behavior of the Chainlink Keeper network. It utilizes `vm.warp` to fast-forward time, proving that `checkUpkeep()` only returns `true` when a round is either expired or sold out. It further validates that `performUpkeep()` correctly locks the round and transitions the state to `Calculating`.
- **Financial Settlement & Treasury (`VRFAndTreasury.t.sol`):** Tests the pool distribution math and the Oracle callback logic. It proves that upon winning, the `prizePool` is correctly split—routing the 2% protocol fee to the `treasury` and the remainder to the winner's vault. It also validates the "rollover" math, ensuring that if the VRF selects an unowned ticket index, the contract autonomously iterates the bitmap to find the nearest valid winner.
- **Liveness Guards & Safety (`Refunds.t.sol`):** Validates the trustless fallback system. It simulates a scenario where the oracle fails to respond, proving that after the `DRAW_TIMEOUT` has passed, the `enableRefundFallback()` function allows the round to become `Refundable`. This ensures that user funds can never be permanently trapped in a stalled draw cycle.

5.5 Mathematical Bias & Statistical Modules

To prove the fairness of the cryptographic primitives and the robustness of the accounting logic at scale, specific statistical and high-concurrency test modules were engineered.

- **Randomness Distribution (`test/security/BiasTest.t.sol`):** This module utilizes a Monte Carlo simulation (`test_MathematicalDistributionBias`) executing 30,000 iterations to validate the `keccak256` modulo arithmetic. By asserting that winning outcomes distribute across participants with a variance threshold of less than 3%, we programmatically prove that the deterministic selection is statistically indistinguishable from uniform randomness. Additionally, `test_LastMinuteSnipeDoesNotBreakMath` confirms that entries recorded immediately before `closeSale()` do not disrupt the distribution formula or the phase state.
- **High-Concurrency & Accounting (`test/security/StressTest.t.sol`):** This module evaluates the protocol's behavior under extreme load and anomalous inputs.
 - **EVM Saturation:** The `test_MassiveConcurrentBuyers` test simulates a "worst-case" scenario by generating unique addresses for up to `MAX_TICKETS` (100) and executing simultaneous purchases. This proves that the `address(this).balance` perfectly matches the aggregate `TICKET_PRICE` without state corruption.
 - **Property-Based Fuzzing:** We utilize Foundry's fuzzing engine in `testFuzz_RevertIfOverpayment` to generate arbitrary ETH values, ensuring that any payment deviating from the strict ticket price is rejected, thus protecting the prize pool's mathematical integrity.

- **Solvency Invariants:** The `invariant_ContractBalanceNeverBreaks` function serves as a persistent property check. It asserts that the contract's raw ETH balance must always remain greater than or equal to the calculated `participantCount * TICKET_PRICE`, guaranteeing that the protocol remains solvent regardless of the number of state mutations.

5.6 Property-Based Fuzzing (test/echidna/)

To stress-test the protocol beyond the reach of manual unit tests, we utilized **Echidna**, a Haskell-based property fuzzer. This process required engineering specialized wrapper contracts that inherit the target logic and expose boolean invariants as public functions. A total of over **3,000,000 fuzzing sequences** (1 million per tier) were executed, failing to find a single counter-example to our core safety properties.

- **Base Protocol Invariants (EchidnaLottery.t.sol):** Executed 1,001,722 fuzzing runs to verify foundational state integrity. This wrapper validated `echidna_balanceCoversPrizePool` (guaranteeing contract solvency), `echidna_validPhaseBounds` (restricting the state machine to valid enum indices), and `echidna_participantsUnderLimit` (proving the `maxTickets` cap is unbreachable). The fuzzer reached 2,936 unique EVM instructions with zero failures.
- **Oracle Lifecycle Invariants (EchidnaLotteryVRF.t.sol):** Validated the V2 contract with 1,001,784 runs. This suite focused on identical solvency properties while enforcing tighter VRF-specific phase bounds (indices 0 to 2), successfully reaching 2,236 unique instructions.
- **Autonomous Engine Invariants (EchidnaLotteryEX.t.sol):** This represented our most complex fuzzing target, reaching over 5,000 unique instructions across 1,002,039 runs. By injecting mock coordinator addresses in the constructor, we isolated the engine's internal logic. We programmatically verified `echidna_ticketsSoldLimit` to physically guarantee the bitmapped storage can never exceed its 256-bit capacity. Additionally, `echidna_validRoundId` proved the auto-incrementing round counter remains corruption-free even under aggressive, random state-mutation attempts.

5.7 Project Configuration & Infrastructure Files

The repository root contains critical configuration files that enforce strict environmental consistency across the compilation, symbolic analysis, and fuzzing phases.

- **foundry.toml & foundry.lock:** These dictate the Paradigm toolchain's behavior. We explicitly configured high-intensity testing parameters, including 10,000 runs for both fuzzing and invariants with a traversal depth of 500. The compiler is pinned to the `paris` EVM version to ensure stability across forks. The lockfile ensures cryptographic determinism for upstream dependencies, specifically pinning OpenZeppelin v5.6.1 and Chainlink Brownie contracts.
- **echidna.yaml:** This custom configuration governs the Haskell-based fuzzer. It enforces a `testLimit` of 1,000,000 sequences and a `seqLen` of 500, distributing the heavy computational load across 8 multi-core `workers` to maximize instruction coverage within a reasonable time window.
- **mythril_solc.json:** This configuration file is vital for the symbolic execution engine. It defines the exact library remappings required to resolve OpenZeppelin and Chainlink imports during the SMT solving phase. It also enables the Solidity optimizer (200 runs) to ensure the analyzed bytecode matches the production deployment.

5.8 CI/CD Pipeline & GitHub Actions

To maintain "Continuous Security," the Veritas repository utilizes GitHub Actions to execute automated audit workflows on every push and `pull_request`.

- **Continuous Integration (test.yml):** An automated pipeline that initializes the Foundry environment, verifies code formatting (`forge fmt`), builds the project to check for compiler errors, and executes the entire test suite with maximum verbosity (`-vvv`).

- **DevSecOps & Gas Audit (audit.yml):** A specialized security workflow that goes beyond standard testing. It programmatically generates gas snapshots to detect regressive optimizations and executes `forge coverage` to enforce the 70%+ line coverage mandate. Finally, it triggers the `Slither` static analysis action to scan for common vulnerabilities (reentrancy, uninitialized storage) before any code is merged into the main branch.

6 Gas Optimisation & EVM Efficiency

Smart contract execution costs are primarily driven by storage operations (SSTORE and SLOAD). The Veritas protocol was engineered to minimize these costs through aggressive EVM-level optimizations, moving from the academic baseline of `Lottery.sol` to the enterprise-grade efficiency of `LotteryEX.sol`. All metrics below are derived from the median execution costs provided by the `forge test -gas-report` utility.

6.1 Optimization 1: Bytecode Injection (immutable Variables)

In the initial development phase, the ticket price was stored as a standard state variable, requiring a 100-gas "warm" SLOAD for every purchase verification. By refactoring this to an `immutable` type, the value is embedded directly into the contract's execution bytecode.

- **Base (Standard Variable):** ~25,900 gas (estimated)
- **Base (immutable Variable):** 23,822 gas (Median)
- **Net Savings:** ~2,000 gas per transaction.
- **Total Impact:** Over a 256-ticket round, this saves users approximately **512,000 gas**.

6.2 Optimization 2: O(1) Bitmapped Storage

The flagship optimization in `LotteryEX.sol` is the elimination of dynamic arrays for tracking ticket ownership.

- **The Vulnerability:** `Lottery.sol` utilizes `participants.push()`, which requires an SSTORE to update the array length and another to write the data, scaling linearly with the number of tickets.
- **The Optimization:** `LotteryEX.sol` uses a single `uint256 ticketBitmap`. A bitwise OR operation (`|`) flips a specific bit to mark a ticket as sold.
- **Impact:** This keeps the data footprint permanently locked to a single 32-byte storage slot. While the median cost for `batchBuyTickets` in `LotteryEX` is 145,241 gas, this represents a massive batch of up to 256 tickets. The per-ticket storage overhead is reduced by **99.6%** compared to individual array pushes.

6.3 Optimization 3: EVM Struct Packing (The 2-Slot Engine)

We mathematically structured the `Round` struct in `LotteryEX` to occupy exactly two 32-byte EVM storage slots, preventing "slot-bloat."

- **Slot 0:** `uint256 ticketBitmap` (32 bytes).
- **Slot 1:** `address winner` (20 bytes) + `uint16 ticketsSold` (2 bytes) + `Phase phase` (1 byte).

Because Slot 1 totals only 23 bytes, the EVM packs these variables together. Updating the `ticketsSold` and the `phase` simultaneously during a draw now costs a single SSTORE execution (5,000 gas if warm) instead of two.

6.4 Optimization 4: Custom Errors vs. Require Strings

To reduce deployment costs and execution overhead, all contracts utilize Solidity 0.8.4+ Custom Errors. This prevents the storage of long ASCII strings in the bytecode.

- **Base (Strings):** 8,810 Bytes (Deployment Size)
- **Extended (Custom Errors):** 15,816 Bytes (Deployment Size)

Despite the LotteryEX contract including significantly more complex logic (Proxies, VRF v2.5, Keepers, and Treasury routing), the use of Custom Errors kept the contract size well below the 24.5kb EIP-170 limit.

6.5 Optimization 5: Memory Caching and Fail-Fast Logic

In the `fulfillRandomWords` and `checkUpkeep` functions of LotteryEX, we implemented two high-level execution optimizations:

- **Memory Caching:** In `fulfillRandomWords`, state variables like `round.prizePool` are aggressively cached into local memory pointers. This allows the contract to perform math and emit events by reading from memory (3 gas) rather than performing repeated SLOAD operations (100 gas).
- **Fail-Fast Keepers:** The `checkUpkeep` function evaluates the phase enum (packed in Slot 1) as the very first line of logic. If the round is not in the `Open` phase, the function aborts instantly. This saves the Chainlink Automation network from simulating expensive timestamp or ticket-count logic, reducing the protocol's operating premiums.

Table 1: Foundry Gas Report - Median Execution Costs

Function Name	Min (Gas)	Median (Gas)	Max (Gas)	# Calls
<code>Lottery.buyTicket</code>	23,800	23,822	93,616	383
<code>Lottery.batchBuyTickets</code>	24,179	56,537	2,445,889	8
<code>LotteryEX.batchBuyTickets</code>	2,830	145,241	5,976,581	27
<code>Lottery.revealAndDraw</code>	24,339	60,957	61,005	12
<code>Lottery.claimPrize</code>	29,128	33,421	37,472	9
<code>LotteryEX.withdrawPrize</code>	17,953	18,104	24,895	3

6.6 Contract Size Profiling (EIP-170 Compliance)

Ethereum's Spurious Dragon hard fork (EIP-170) introduced a strict smart contract size limit of 24.576 KB. Exceeding this limit results in a deployment failure (OOG error). As part of our DevSecOps pipeline, we utilized `forge build -sizes` to ensure our architectures, particularly the feature-dense Tier 3 engine, remained compliant with mainnet constraints.

Through our aggressive use of custom errors, bitmapped storage, and perfect struct packing, even our most complex engine (LotteryEX) utilized only approximately 62% of the maximum allowed bytecode, leaving significant margin for future governance or logic expansions.

Table 2: Contract Size Profiling (EIP-170)

Contract	Runtime Size (B)	Initcode Size (B)	Runtime Margin (B)
Lottery	7,905	8,746	+16,671 B
LotteryVRF	5,907	7,143	+18,669 B
LotteryEX	15,207	15,816	+9,369 B

7 Test Coverage, Fuzzing & Invariants

A highly resilient Web3 protocol requires absolute mathematical certainty. The Veritas test suite achieved a flawless **100% Line, Statement, and Branch coverage** across all core smart contract architectures, successfully navigating complex mathematical rollover loops and bitmapped bounds checking.

File	% Lines	% Statements	% Branches
src/Lottery.sol	100.00% (54/54)	100.00% (53/53)	100.00% (33/33)
src/LotteryEX.sol	100.00% (115/115)	100.00% (138/138)	100.00% (19/19)
src/LotteryVRF.sol	100.00% (31/31)	100.00% (35/35)	100.00% (9/9)
Total (Core logic)	100.00%	100.00%	100.00%

Listing 4: Forge Coverage Report - Core Source Contracts

7.1 Massive Concurrent Stress Testing

To prove that the EVM accounting logic holds under extreme transaction loads, the `test_MassiveConcurrentBuyers` suite simulated 100 distinct wallet addresses purchasing tickets simultaneously to reach the `MAX_TICKETS` cap. The execution consumed **3,415,598 gas** in the simulation environment, mathematically proving that the final contract balance perfectly matched the aggregate ticket revenue. This test guarantees no accounting drift, rounding errors, or state corruption occurs even when the EVM storage is fully saturated.

7.2 Statistical Bias Analysis (Monte Carlo)

A Monte Carlo simulation within `BiasTest.t.sol` executed 30,000 algorithmic draw iterations to analyze the modulo arithmetic dispersion of the V1 architecture (`uint256(keccak256(...)) % participants.length`). The results proved an even distribution with an acceptable variance bound of less than 3%, confirming algorithmic fairness. Furthermore, the `test_LastMinuteSnipeDoesNotBreakMath` execution proved that purchasing a ticket in the exact same block as the `closeSale` transaction yields zero mathematical advantage to the attacker.

7.3 Property-Based Fuzzing & Invariants

Property-based fuzzing ensures that critical system constraints (invariants) cannot be broken, regardless of arbitrary, randomized, or malicious input data payloads.

- **Foundry Invariants:** The `invariant_ContractBalanceNeverBreaks` property was subjected to **5,000,000 recursive calls** across 10,000 sequence runs. It successfully proved that malicious input data can never force the contract into insolvency or bypass the Checks-Effects-Interactions (CEI) guardrails.
- **Echidna (Haskell Fuzzer):** Using the containerized Echidna tool configured via `echidna.yaml` (maxed to 1,000,000 sequences across 8 CPU workers), strict boundary rules were mathematically verified against the EVM bytecode:
 - **V1 Base Lottery:** Verified foundational solvency across **2,936 unique instructions** (Seed: 4338489265008950013).
 - **V2 VRF Lottery:** Verified `echidna_validPhaseBounds` across **2,236 unique instructions** with a generated corpus size of 24 (Seed: 8033127568757907724).
 - **V3 LotteryEX:** This represented the deepest analysis, reaching **5,027 unique instructions**. It specifically proved `echidna_ticketsSoldLimit`, verifying that the bitmapped storage engine can never overflow its 256-bit capacity (Seed: 671137308507522134).

8 Live Network Validation & Deployment Artifacts

To demonstrate the protocol's readiness for production environments, we executed successful deployments on both a local development network (Anvil) and the public Ethereum Sepolia testnet. This validates the pipeline's ability to handle varied RPC providers, gas price volatility, and cryptographic signing.

8.1 Local Network Validation (Anvil)

The following logs capture the successful execution of `Deploy.s.sol` on a local fork, used to benchmark base deployment gas costs before committing funds to a public network.

```
Script ran successfully.
Gas used: 1,716,425

== Return ==
0: contract Lottery 0x7FA9385bE102ac3EAc297483Dd6233D62b3e1496

== Logs ==
Lottery deployed at: 0x7FA9385bE102ac3EAc297483Dd6233D62b3e1496
Ticket Price (wei): 10,000,000,000,000,000 (0.01 ETH)
Max Tickets: 100
```

Listing 5: Foundry Anvil Deployment Logs

8.2 Ethereum Sepolia Testnet Deployment

The protocol was successfully broadcast to the Ethereum Sepolia network (Chain ID: 11155111). The deployment utilized an EIP-1559 transaction to initialize the `Lottery` state machine on a public ledger.

- **Contract Address:** `0x7bf05362b4e802878257c77b7d748ac4a4768016`
- **Transaction Hash:** `0x710d54a2bffa86140e6b499b3ae194451fb7888ba15f25ae33f1a33ad655d1c`
- **Actual Gas Used:** 1,829,656 units
- **Network Status:** Confirmed (Status 0x1)

```
{
  "transactionType": "CREATE",
  "contractName": "Lottery",
  "contractAddress": "0x7bf05362b4e802878257c77b7d748ac4a4768016",
  "args": ["10000000000000000", "100"],
  "hash": "0x710d54a2b...33ad655d1c",
  "chainId": "11155111"
}
```

Listing 6: Sepolia Deployment Metadata (Broadcast JSON Summary)

9 Conclusion

The Veritas project successfully demonstrates the evolution of decentralized logic from rudimentary trust models to fully autonomous, oracle-verified engines. By enforcing a 100% test coverage mandate, executing millions of fuzzed transaction sequences, and containerizing the DevSecOps audit pipeline, the protocol achieves military-grade security while optimizing gas costs through advanced EVM storage manipulation. The successful deployment to the Sepolia testnet serves as the final validation of the protocol's architectural integrity and its readiness for the Ethereum mainnet.